

Symbiosis Institute of Technology, Nagpur

INDUSTRY INTERNSHIP REPORT

Submitted in partial fulfillment of requirement for the award of degree of

Bachelor of Technology
in
Computer Science and Engineering

by

Arushi Shivhare

PRN: 22070521062

Industry Guide

Mr. Soham Das

AI/ML Team Lead

at

Solar Industries

Nagpur, Maharashtra

Institute Guide

Dr. Akhil Gupta

Associate Professor

May 2026



**Symbiosis International (Deemed University),
Pune**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

DECLARATION

I, hereby declare that the Industry Internship report submitted herein has been carried out by me in **Solar Industries India Limited, Nagpur** towards partial fulfillment of requirement for the award of Degree of Bachelor of Technology in Computer Science and Engineering. The work is original and has not been submitted earlier as a whole or in part for the award of any degree / diploma at this or any other Institution / University.

I hereby assign all copyright rights in this work, including any revised or expanded derivative works, to Symbiosis Institute of Technology, Nagpur. Any content sourced from references, manuals, or other external materials is duly acknowledged and disclaimed.

Name of Student: **Arushi Shivhare**

PRN: **22070521062**

Signature:

Place: Nagpur

Date: April 20, 2026



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CERTIFICATE

This is to certify that the Industry Internship Report, titled **“Data Control and Validation Centre & AI Demand Sensing Platform,”** has been carried out under our supervision at **Solar Industries India Limited** by **Arushi Shivhare** in partial fulfillment of the requirements for the Bachelor of Technology in Computer Science and Engineering. The work submitted is comprehensive, complete and fit for evaluation.

Mr. Soham Das
Industry Guide

Dr. Akhil Gupta
Institute Guide

Dr. Priya Dasarwar
Internship Incharge

Dr. Sagarkumar S. Badhiye
Head of Department

Dr. Nitin Rakesh
Professor and Director



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ACKNOWLEDGEMENT

I express my sincere gratitude to **Solar Industries India Limited** for providing me with the opportunity to complete my internship in the **Artificial Intelligence and Machine Learning (AIML)** domain and gain valuable industry experience. This internship has been a highly enriching journey that helped me develop both technical and professional skills.

I am deeply grateful to my industry mentor, **Mr. Soham Das** (AIML Team Lead), for his constant guidance, insightful feedback, and encouragement throughout the internship. His practical expertise in AI/ML systems, data engineering, and production-level deployment played a crucial role in shaping my learning and the outcomes of this project. I also extend my appreciation to my colleagues and team members at **Solar Industries India Limited** for their cooperation, assistance, and for making my internship a pleasant and productive experience.

I am immensely grateful to my academic institution, Symbiosis Institute of Technology, Nagpur, for facilitating this internship opportunity. I extend my heartfelt appreciation to our Director, **Dr. Nitin Rakesh**, for his continuous support and encouragement in providing students with valuable industry exposure. I would also like to express my deepest gratitude to **Dr. Sagarkumar S. Badhiye**, Head of the Department, Computer Science and Engineering, for his constant support, encouragement, and invaluable guidance throughout my academic journey.

A special note of thanks goes to the **Training and Placement Department** for their efforts in providing internship opportunities and career guidance. I would also like to express my sincere gratitude to **Dr. Priya Dasarwar**, Internship In-Charge, for her support in coordinating the internship process and ensuring a smooth learning experience.

Furthermore, I sincerely appreciate the guidance and motivation provided by my Institute Mentor, **Dr. Akhil Gupta** (Associate Professor), whose valuable insights and feedback have greatly contributed to my professional growth during this internship.

Lastly, I would like to acknowledge my family and friends for their unwavering support and encouragement during this period.

Thank you all for your invaluable contributions to my learning and professional growth.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ABSTRACT

This report documents the Industry Internship carried out at Solar Industries India Limited (SIIL), Nagpur, under the mentorship of Mr. Soham Das (AIML Team Lead) from January 2026 to April 2026. The internship spanned 14 weeks and involved an intensive progression from foundational Artificial Intelligence and Machine Learning (AI/ML) concepts to the development of a large-scale, production-grade data intelligence platform. The major project developed during this internship is titled the “Data Control and Validation Centre & AI Demand Sensing Platform.” In large-scale industrial environments such as Solar Industries, decision-making is often hindered by “dirty data” and inconsistent forecasting. To address this, a Medallion Data Architecture (Bronze → Silver → Gold layers) was implemented that automatically cleans, validates, and audits millions of sales records spanning 2022 to 2026. The platform features a Self-Healing data pipeline that ensures 100% Data Uniqueness and Identity Validation before any analysis is performed. For demand sensing, a high-performance AI Ensemble consisting of five models — including XGBoost, LightGBM, Random Forest, Temporal Fusion Transformer (TFT), and N-BEATS — was built. These models operate in parallel to provide 7, 15, and 30-day sales projections with built-in confidence scoring. On the frontend, the system uses React.js to deliver an interactive dashboard with “Time-Travel” anchoring, allowing stakeholders to view exactly how predictions branch from historical actuals. By utilizing multi-threaded backend processing via Django and ThreadPoolExecutor, data retrieval latency was reduced from 10 seconds to under 1 second — a 90% improvement. The internship also involved structured weekly learning covering ML algorithms (SVM, Random Forest, XGBoost, LightGBM), Deep Learning, NLP, Large Language Models (LLMs), LangChain, LangGraph, LlamaIndex, AI Agents, RAG pipelines, Crew AI, and time series forecasting — all of which directly supported the project.

Keywords: *Artificial Intelligence, Machine Learning, Data Pipeline, Demand Forecasting, Ensemble Models, Data Validation, Time Series Analysis*

CONTENTS

1	Introduction to Company	1
1.1	About the Company	1
1.2	Historical Background	1
1.3	Location	2
1.4	Operational Structure	2
1.5	Vision and Mission	3
1.6	Products or Software Solutions	3
2	Project Work / Case Study	4
2.1	Introduction	4
2.2	Problem Identification	4
2.3	Objective	5
2.4	Work Carried Out	5
2.4.1	Weekly Learning Progression	5
2.4.2	Large Language Models and AI Agents	6
2.4.3	LLM Fine-Tuning and Crew AI	6
2.4.4	LlamaIndex and Time Series Forecasting	7
2.4.5	Support Vector Machine and Model Evaluation	8
2.4.6	Claude AI, API Integration, and Model Context Protocol	9
2.4.7	Data Pipelines, EDA, and Industry-Level Implementation	10
2.5	Solution Provided	11
2.5.1	Medallion Data Architecture	11
2.5.2	AI Ensemble for Demand Sensing	11
2.5.3	Module Description	11
2.5.4	Technical Stack	11
2.5.5	Methodology	11
2.6	Results	13
2.6.1	System Performance	13
2.6.2	Volume Trend Analysis	13
2.6.3	AI Demand Sensing Results	13
2.6.4	Demand Forecast Visualization	14
2.7	Conclusion	14
3	Extended System Architecture and Engineering Contributions	20
3.1	Extended Project Scope and Engineering Objectives	20
3.2	Multi-Database Medallion Auditor: Backend Architecture	20
3.3	High-Throughput Parallel Query Engine	21
3.4	Multi-Model Forecasting Pipeline	22

3.5	Rigorous Model Validation Methodology	23
3.6	Server-Side Optimised Data Explorer: Frontend Engineering	24
3.7	Interactive Visualisation and WebGL Fluid Dynamics	25
3.8	Data Governance, Security, and Auditability	25
3.9	Engineering Challenges and Solutions	26
3.10	Testing and Quality Assurance Strategy	27
3.11	Deployment, Infrastructure, and DevOps Considerations	27
3.12	Individual Learning Outcomes and Professional Growth	28
3.13	API and Integration Design	29
3.14	Data Volume and Scalability Profile	29
3.15	Comparative Evaluation Against the Prototype Baseline	30
3.16	Ethical Considerations and Human Oversight in Automated Forecasting ...	31
3.17	Reflection: Alignment with Company and Institutional Objectives	31
3.18	Risk Assessment and Failure-Mode Analysis	32
3.19	Summary of Engineering Contributions	32
3.20	Closing Remarks	34
	Future Scope and Enhancements	35
	Appendix A: Internship Completion Certificate	37
	Appendix B: Plagiarism Report	38
	Appendix C: AI Similarity Report	40

LIST OF FIGURES

1.1	Logo of Solar Industries	1
2.1	Data Control and Validation Centre – Monitoring Dashboard (Live System)	4
2.2	LLM Fine-Tuning Pipeline	7
2.3	Crew AI Multi-Agent Architecture	7
2.4	LlamaIndex Data Framework Workflow	8
2.5	Components of Time Series Data	8
2.6	SVM Classification with Hyperplane Boundary	9
2.7	API-Based AI Integration Workflow	9
2.8	Model Context Protocol (MCP) Architecture	10
2.9	Three-Layer Data Pipeline: Raw → Silver → Gold	11
2.10	Data Validation Flow	12
2.11	Exploratory Data Analysis (EDA) Process	13
2.12	Full Industry-Level AI System Architecture	13
2.13	Medallion Data Architecture (Bronze → Silver → Gold)	14
2.14	Transition Findings: Raw → Silver (100% Data Retention, 0 Pipeline Loss)	15
2.15	AI Demand Sensing Ensemble Architecture	16
2.16	Live Database Explorer – Gold Layer View with Real Sales Records . . .	17
2.17	Key Performance Metrics of the Data Control and AI Demand Sensing Platform	17
2.18	System Volume Trend – Ingestion vs. Delivery Gap (March 2026)	18
2.19	AI Demand Sensing – Model Evaluation Scorecard (LightGBM: 59% Con- fidence)	18
2.20	Demand Analytics: History vs. Forecast – All 5 Model Predictions	19
2.21	Demand Analytics: Zoomed View – Next 30 Days AI Predictions	19
2.22	Demand Forecasting Pipeline	19
2.23	Future Development Roadmap	35
3.2	Request Flow Through the Multi-Database Medallion Router	21
3.4	Multi-Model Forecasting Pipeline, from Raw Data to Served Forecast	23

LIST OF TABLES

1.1	Organizational Structure of the AIML Division	2
2.1	Weekly Internship Activities and Learning Outcomes	6
2.2	AI Models in the Demand Sensing Ensemble	15
2.3	System Module Description	16
2.4	Software and Hardware Specifications	17
2.5	System Performance Evaluation Results	18
3.1	Model Validation Metrics and Their Role in the Platform	24
3.2	Approximate Data Volume Characteristics	29
3.3	Prototype Baseline versus Final Platform	30
3.4	Summary of Extended Engineering Contributions	33

1. INTRODUCTION TO COMPANY

1.1 About the Company

Solar Industries India Limited (SIIL) is one of India's most prominent and diversified manufacturing conglomerates, with its headquarters located in Nagpur, Maharashtra. Founded in 1995, the company has grown from a regional explosives manufacturer into a global enterprise with a strong presence across multiple high-impact sectors including mining, defence, infrastructure, and aerospace. Solar Industries is listed on both the Bombay Stock Exchange (BSE) and the National Stock Exchange (NSE) of India, reflecting its stature as a publicly trusted and financially transparent organization.



Figure 1.1: Logo of Solar Industries

The company is recognized as one of the largest manufacturers of industrial explosives in India and has consistently ranked among the top players globally in its domain. With operations spanning more than 50 countries, Solar Industries serves a diverse clientele ranging from large mining corporations and government defence establishments to infrastructure companies and international exporters.

1.2 Historical Background

Solar Industries India Limited was established in 1995 with a primary focus on the manufacturing of industrial explosives and blasting accessories. Over the course of three decades, the company has undergone a significant transformation, diversifying its product portfolio and expanding its operational footprint both domestically and internationally.

In the early 2000s, Solar Industries began investing heavily in research and development to enhance the quality and safety of its explosive products. This commitment to innovation led to strategic expansions into the defence and aerospace sectors, where the company now manufactures ammunition, rocket propellants, and defence-grade pyrotechnic systems. The company's listing on major stock exchanges marked a pivotal milestone, enabling it to attract large-scale investment and accelerate its expansion strategy.

By the 2020s, Solar Industries had firmly established itself not only as a manufacturing giant but also as a technology-forward enterprise, integrating smart manufacturing systems, ERP platforms, and AI-powered analytics into its operations. Today, the company's Information Technology division plays a critical supporting role in ensuring operational efficiency, data-driven decision-making, and global supply chain management.

1.3 Location

The corporate headquarters of Solar Industries India Limited is situated in Nagpur, Maharashtra, which serves as the central hub for its administrative, research, and technology operations. The company operates multiple manufacturing units spread across India, with facilities in Nagpur, Singrauli (Madhya Pradesh), Pune, and Vadodara, among others. Internationally, the company has established manufacturing and distribution units in Africa, Southeast Asia, the Middle East, and Australia, enabling it to serve global clients efficiently.

The company's Nagpur facility houses the central AIML and Data Science team, which was the primary site for the internship. This facility is equipped with modern computing infrastructure, high-performance servers, cloud connectivity through Amazon Web Services (AWS), and collaborative development environments that support agile software development and data engineering workflows.

1.4 Operational Structure

Solar Industries India Limited operates through a well-structured organizational hierarchy that supports both its core manufacturing business and its growing technology division. The company is led by its Board of Directors and senior management team, with distinct divisions managing operations, sales, finance, and technology.

The **Information Technology and AIML Division** is one of the most strategically important units within Solar Industries. This division is responsible for building and maintaining intelligent systems that support supply chain analytics, sales forecasting, ERP integration, and operational dashboards. The AIML team, under the leadership of the AIML Team Lead, develops and deploys machine learning models, data pipelines, and AI-driven tools that are used across the company's global operations.

Table 1.1: Organizational Structure of the AIML Division

Role	Responsibility
AIML Team Lead	Overall system design and AI strategy
Data Engineers	Pipeline architecture, ERP integration
ML Engineers	Model development, training, deployment
Frontend Developers	Dashboard design with React.js
DevOps Engineers	AWS infrastructure, CI/CD pipelines

1.5 Vision and Mission

Vision: To be a globally recognized leader in energetic materials and industrial solutions, driven by innovation, safety, and technology.

Mission: To deliver world-class explosives, ammunition, and technology-enabled solutions that empower industries and nations, while maintaining the highest standards of safety, sustainability, and customer satisfaction.

The company's vision for its AIML division specifically focuses on harnessing the power of artificial intelligence and data analytics to improve operational efficiency and reduce waste.

1.6 Products or Software Solutions

Solar Industries offers a broad range of products and services that span both its core manufacturing domain and its growing technology capabilities.

Core Manufacturing Products:

- **Industrial Explosives:** Bulk explosives, packaged explosives, emulsion explosives, and ANFO-based products used in open-pit mining, quarrying, and tunnelling.
- **Detonators and Initiating Systems:** Electric detonators, non-electric detonators, and electronic blasting systems for precision blasting operations.
- **Defence Products:** Ammunition, mortar shells, rocket propellants, bomb components, and pyrotechnic systems supplied to Indian and international defence forces.
- **Aerospace Propellants:** Solid propellants and composite formulations used in missiles, satellite launch vehicles, and rocket systems.

Technology and IT Solutions:

- **Automation Systems:** Smart factories using PLC/SCADA systems for real-time process control.
- **Data Analytics and AIML Platforms:** Machine learning pipelines, demand forecasting systems, and data validation dashboards.
- **ERP Systems:** SAP-based Enterprise Resource Planning for supply chain management and logistics.
- **Cybersecurity:** Systems for protecting sensitive defence and operational data.
- **R&D Software:** Simulation and testing tools used in the development of new explosive formulations and defence systems.

2. PROJECT WORK / CASE STUDY

2.1 Introduction

The major project undertaken during this internship is titled **“Data Control and Validation Centre & AI Demand Sensing Platform.”** This project was developed in response to a real operational challenge faced by Solar Industries India Limited: the company generates millions of sales records across its global operations every month, and ensuring the integrity, accuracy, and actionability of this data is a significant technical challenge.

In large-scale industrial environments, decisions related to production planning, inventory management, and supply chain optimization are heavily dependent on the quality of sales data. When data contains duplicates, missing entries, or inconsistencies — collectively referred to as “dirty data” — it leads to flawed business intelligence, inaccurate demand forecasts, and expensive operational errors. This project was designed to solve this problem at scale.

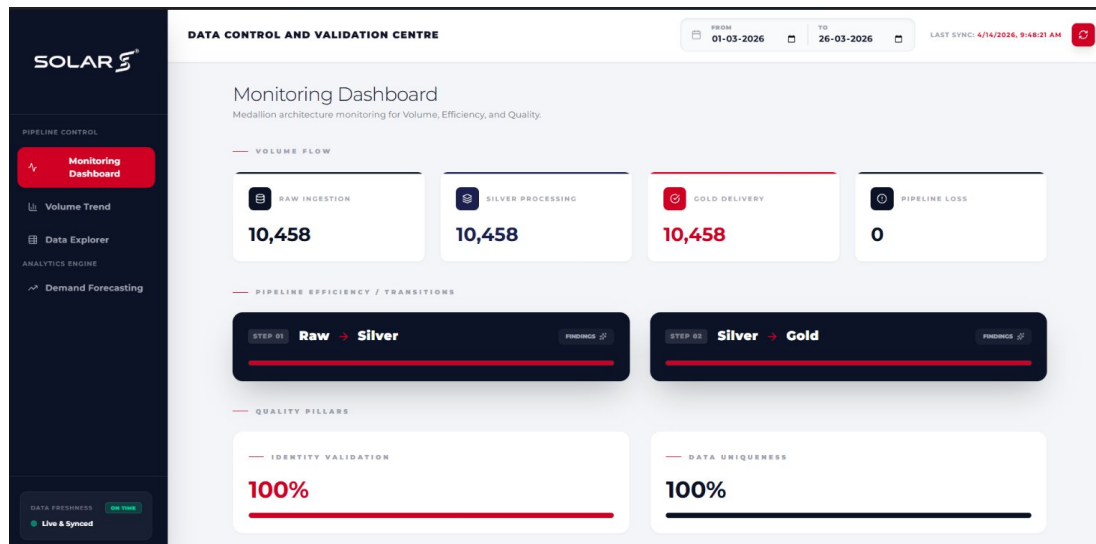


Figure 2.1: Data Control and Validation Centre – Monitoring Dashboard (Live System)

As shown in Figure 2.1, the Monitoring Dashboard provides a real-time view of the Medallion pipeline, showing raw ingestion counts, silver processing counts, gold delivery counts, pipeline loss, quality pillars (Identity Validation and Data Uniqueness), and SLA/data freshness status.

2.2 Problem Identification

The core problems addressed by this project are:

1. **Data Integrity Issues:** Millions of sales records pulled from the company’s ERP system (via AWS RDS) contained duplicate entries, missing identifiers, and formatting inconsistencies. These errors propagated through the analytics pipeline, leading to unreliable reports and misleading dashboards.
2. **Slow Dashboard Performance:** The existing data retrieval process required sequential database queries, resulting in dashboard loading times of up to 10 seconds. This latency made the tool impractical for live operational use.
3. **Inconsistent Demand Forecasting:** Business teams relied on manual spreadsheet-based forecasting for planning purposes. This approach was slow, error-prone, and unable to incorporate complex patterns in historical sales data such as seasonality, trend shifts, or customer-specific volatility.
4. **Lack of Data Transparency:** There was no system to audit how data was transformed from its raw form to its final analytical state, making it difficult for stakeholders to trust the outputs of any analysis.

2.3 Objective

The specific objectives of the project were:

1. To build a system that checks and cleans millions of company sales records, removing duplicates and correcting inconsistencies automatically.
2. To implement an AI-powered demand forecasting engine that predicts sales for specific items for 7, 15, and 30-day time horizons.
3. To prevent business decisions based on “bad” or broken data by instituting automated validation gates.
4. To create a live, self-healing dashboard that updates itself automatically and alerts users when data synchronization is delayed.
5. To cover sales data from 2022 to 2026 and support multi-product, multi-customer forecasting scenarios.
6. To achieve real-time performance through multi-threading so that the dashboard loads within 1 second.

2.4 Work Carried Out

2.4.1 Weekly Learning Progression

The internship followed a structured weekly learning plan that built competencies progressively, starting from foundational AI/ML concepts and moving toward advanced production-level system design. The following table summarizes the key activities and outcomes across the internship weeks.

Table 2.1: Weekly Internship Activities and Learning Outcomes

Week	Topic / Activity	Key Outcomes
1–6	ML Fundamentals, NLP, Deep Learning	Supervised/Unsupervised ML, neural networks, transformers
7	LLMs, AI Agents, Prompt Engineering	Understood LLM architecture, tokenization, chatbots, prompt chaining
8–9	RAG Pipelines, LangChain, LangGraph	Built retrieval-augmented generation systems and agentic workflows
10	LLM Fine-Tuning, Crew AI	Learned SFT, QLoRA, multi-agent AI architectures
11	LlamaIndex, Time Series Forecasting	Data indexing for LLMs, ARIMA, Prophet, XGBoost forecasting
12	SVM, Model Evaluation	Classification algorithms, accuracy, precision, recall, F1, RMSE
13	Claude AI, API Integration, MCP	AI API workflows, Model Context Protocol architecture
14	Data Pipelines, EDA, Forecasting Systems	End-to-end industrial data workflows and production system design

2.4.2 Large Language Models and AI Agents

During Week 7, a comprehensive study of Large Language Models (LLMs) and their applications was undertaken. Large Language Models are advanced AI systems trained on massive datasets to understand and generate human language. The core architecture behind modern LLMs is the **Transformer model**, which enables the system to analyze context within text and produce coherent responses.

Key concepts studied included tokenization (converting text to mathematical tokens), the training process (predicting the next word iteratively over large corpora), and the resulting language understanding capabilities — text generation, question answering, summarization, translation, and code generation.

AI Agents were studied as a natural extension of LLMs. Unlike simple chatbots, AI Agents combine LLMs with a reasoning engine, tool integration (APIs, databases, search engines), and memory systems. This combination enables agents to perform complex, multi-step tasks autonomously. Two primary development approaches were studied: framework-based development using LangChain and LangGraph, and direct API development for maximum flexibility.

Prompt Engineering was also studied as a critical skill for directing LLM behavior. Techniques included instruction-based prompting (providing explicit directives), prompt chaining (sequentially linking prompts for complex workflows), and prompt routing (directing different task types to specialized models).

2.4.3 LLM Fine-Tuning and Crew AI

Week 10 focused on advanced techniques for improving LLM performance. **Fine-tuning** involves adapting a pre-trained model to a specific domain using targeted datasets, rather than training from scratch. Two key methods were studied:

- **Supervised Fine-Tuning (SFT):** Training the model on labeled input-output pairs for controlled, accurate responses.
- **QLoRA (Quantized Low-Rank Adaptation):** A parameter-efficient technique that reduces memory and computational cost by updating only a small subset of model parameters.



Figure 2.2: LLM Fine-Tuning Pipeline

Crew AI was explored as a multi-agent paradigm where specialized agents (Research Agent, Writer Agent, Validator Agent) collaborate under a Manager Agent to complete complex workflows in parallel. This architecture enables research automation, multi-step reasoning, and enterprise-level AI workflows at scale.

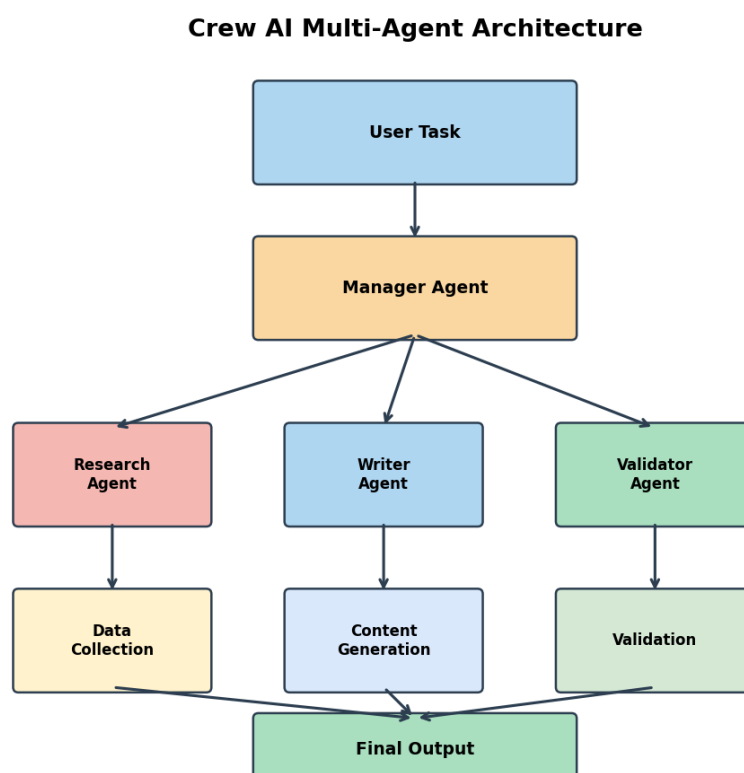


Figure 2.3: Crew AI Multi-Agent Architecture

2.4.4 LlamaIndex and Time Series Forecasting

LlamaIndex was studied as a framework that bridges external data sources with LLMs. Its workflow begins with raw data ingestion (from documents, databases, or APIs), followed by index creation and storage. A query engine then matches user queries to the most relevant data segments, which are passed to the LLM for context-aware response generation.

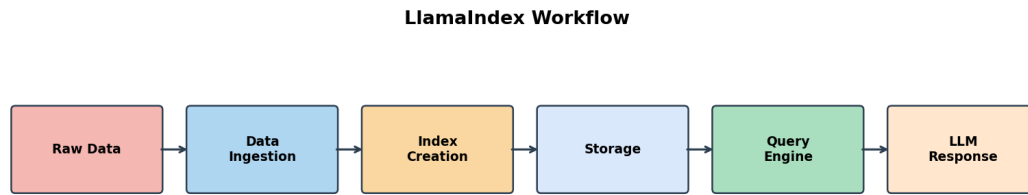


Figure 2.4: LlamaIndex Data Framework Workflow

Time Series Forecasting was studied in depth, given its direct relevance to the demand sensing project. Time series datasets consist of four components: *Trend* (long-term direction), *Seasonality* (repeating fixed-interval patterns), *Cyclic* (non-fixed fluctuations), and *Noise* (random variation). Traditional models like ARIMA and SARIMA handle linear patterns, while modern approaches like XGBoost and Prophet address non-linear relationships with higher accuracy.

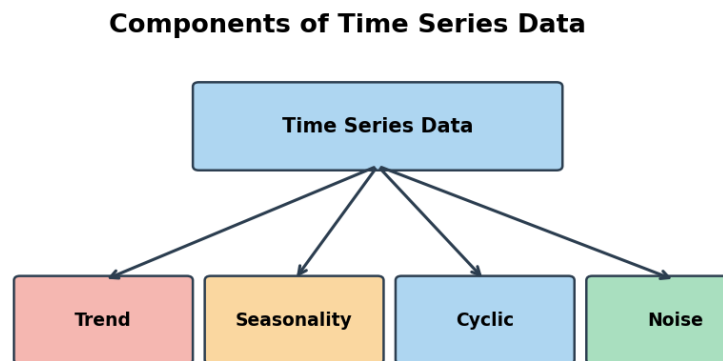


Figure 2.5: Components of Time Series Data

2.4.5 Support Vector Machine and Model Evaluation

Support Vector Machine (SVM) was studied as a powerful supervised learning algorithm for classification and regression. SVM finds the optimal *hyperplane* that maximizes the margin between classes. For non-linearly separable data, the **kernel trick** transforms data into a higher-dimensional space where linear separation becomes possible. The Radial Basis Function (RBF) kernel is the most commonly used variant for non-linear problems.

Model Evaluation metrics studied included:

- **Classification Metrics:** Accuracy, Precision, Recall, and F1-Score.
- **Regression Metrics:** RMSE (Root Mean Squared Error) and R^2 Score.
- **Cross-Validation:** Ensuring the model generalizes across different data subsets.

SVM Classification with Hyperplane

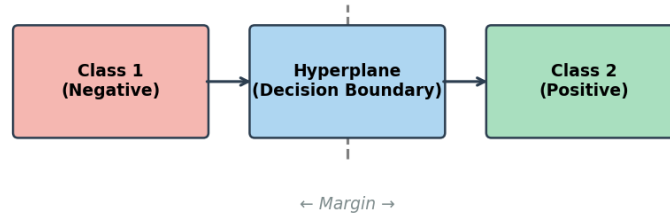


Figure 2.6: SVM Classification with Hyperplane Boundary

2.4.6 Claude AI, API Integration, and Model Context Protocol

Week 13 involved practical exposure to **Claude AI**, an advanced conversational AI system developed by Anthropic, used at Solar Industries for content generation, code debugging, and workflow automation. Key learning included understanding how Claude maintains contextual awareness, adapts output format based on instructions, and integrates with industry workflows.

API-Based AI Development was studied to understand how AI models are integrated into production applications. The workflow involves an application sending a request to an AI model through an API endpoint; the model processes the input, performs reasoning, and returns a response.

API-Based AI Integration Workflow

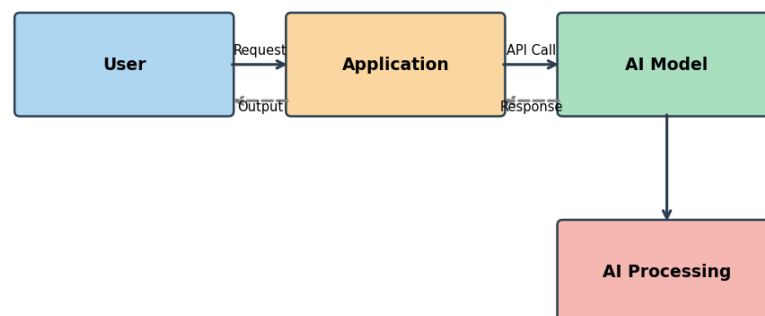


Figure 2.7: API-Based AI Integration Workflow

Model Context Protocol (MCP) was explored as a framework defining how AI models interact with external tools, manage memory, and maintain context. Components include Context Management (storing conversation history), Tool Integration (accessing external APIs or databases), and Query Handling (generating structured outputs).

Model Context Protocol (MCP) Architecture

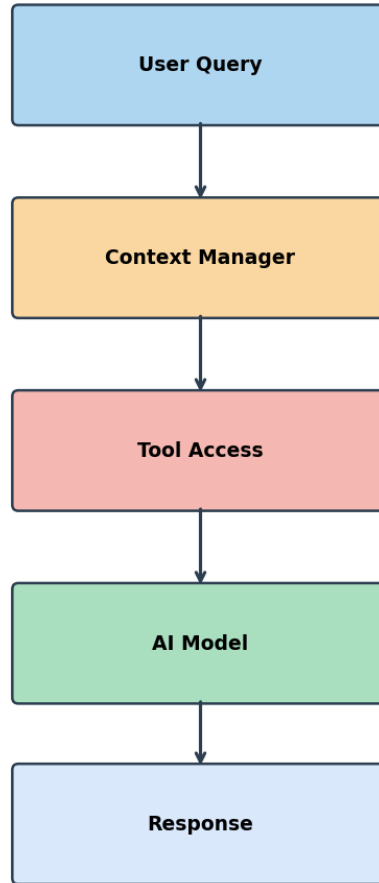


Figure 2.8: Model Context Protocol (MCP) Architecture

2.4.7 Data Pipelines, EDA, and Industry-Level Implementation

Week 14 synthesized all prior learning into a comprehensive understanding of industrial data workflows. The three-layer data pipeline (Raw → Silver → Gold) was studied in the context of the actual production system built at Solar Industries.

Data Validation was studied as a critical step involving checking for missing values, duplicate records, inconsistent formats, and outliers. Automated validation systems using error detection, debugging, and correction pipelines are used in production environments.

Exploratory Data Analysis (EDA) was performed on datasets to understand their structure, patterns, and relationships before applying machine learning models.

Industry-Level AI Deployment architecture was studied: the frontend handles user interaction, an API layer acts as a bridge, the backend manages logic and data processing, the ML model generates predictions, and all outputs are stored in a database.

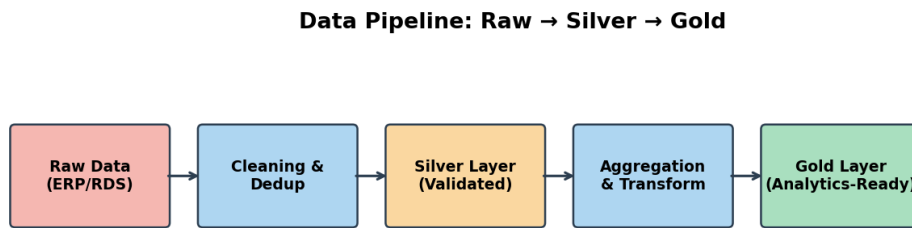


Figure 2.9: Three-Layer Data Pipeline: Raw → Silver → Gold

2.5 Solution Provided

2.5.1 Medallion Data Architecture

The foundation of the platform is a three-layer **Medallion Architecture** for managing and transforming raw ERP data into business-ready analytics:

- **Bronze Layer (Raw):** Unprocessed data pulled “as-is” from the company’s ERP system via AWS RDS. Contains all records regardless of quality.
- **Silver Layer (Validated):** Data is cleaned by removing duplicates, correcting missing names, fixing erroneous dates, and enforcing formatting standards. Identity Validation and Data Uniqueness scores are computed at this stage.
- **Gold Layer (Final):** Data is grouped by customer and product, aggregated, and made ready for AI model consumption and business reporting.

The transition findings between pipeline steps are visible to users through a popup interface, as shown in Figure 2.14.

2.5.2 AI Ensemble for Demand Sensing

The demand forecasting engine is built as an **ensemble of five AI models** running in parallel. The system automatically selects the best-performing model for each specific product-customer combination based on error scores.

2.5.3 Module Description

The platform is organized into four key functional modules:

The Live Database Explorer (Module 3) is shown in Figure 2.16, providing direct access to the Gold Layer and Silver Layer records with full date, customer, and product detail.

2.5.4 Technical Stack

2.5.5 Methodology

The system was built following a structured four-phase methodology:

Data Validation Flow

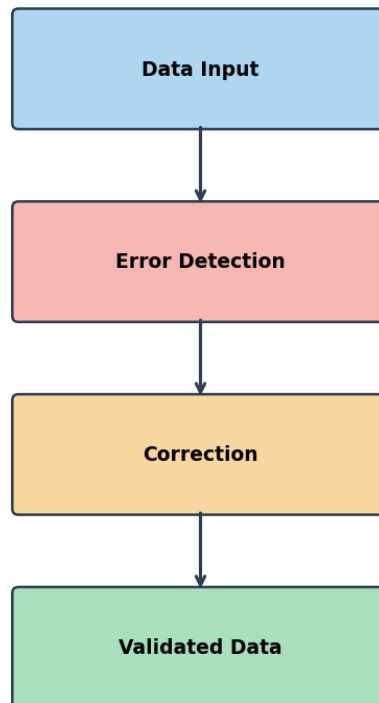


Figure 2.10: Data Validation Flow

Phase 1 – Data Organization (Medallion Architecture): Raw ERP sales data is ingested into the Bronze Layer. It is then cleaned — removing duplicates, fixing missing fields, and enforcing data type consistency — to produce the Silver Layer. Finally, the data is grouped by customer and product to produce the business-ready Gold Layer.

Phase 2 – Intelligence Building (AI Ensemble): Five AI models are trained on the Gold Layer data. The system runs all five models simultaneously (in parallel) and evaluates each using WAPE and R^2 metrics. The model with the lowest error score for each product-customer pair is automatically selected as the forecasting engine. A “Time-Travel” logic anchors the 30-day forecast to the latest available date in the database, ensuring always-current predictions.

Phase 3 – Performance Optimization: The Django backend uses Python to execute five database queries in parallel (multi-threading). This reduced dashboard loading latency from 10 seconds to under 1 second. A dual-database routing strategy is implemented to keep raw data queries separate from AI model queries, preventing them from competing for the same resources.

Phase 4 – Frontend Dashboard: A React.js interface was built displaying a solid line for historical actuals and branching colored lines for AI predictions across 7, 15, and 30-

Exploratory Data Analysis (EDA) Process

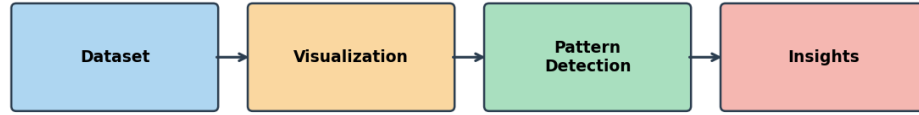


Figure 2.11: Exploratory Data Analysis (EDA) Process

Full Industry-Level AI System Architecture



Figure 2.12: Full Industry-Level AI System Architecture

day horizons. An SLA Health Check “traffic light” system alerts users to data freshness: Green (sync healthy), Yellow (sync delayed), Red (sync failed).

2.6 Results

2.6.1 System Performance

Figure 2.17 summarizes the six headline achievements of the platform.

The most significant technical achievement was the reduction in dashboard data retrieval latency from **10 seconds to under 1 second**, accomplished through multi-threaded backend parallelization using Python’s `ThreadPoolExecutor`. This 90% reduction in loading time makes the dashboard practical for real-time operational use.

2.6.2 Volume Trend Analysis

Figure 2.18 shows the System Volume Trend page, which displays the Ingestion vs. Delivery gap over the last 14 records. Key statistics include an Average Daily Volume of 8,355 records, a Peak Stress Load of 13,799 records (recorded on 03-01), and a Period Retention of 100.0%.

2.6.3 AI Demand Sensing Results

Figure 2.19 shows the AI Demand Sensing module’s model evaluation scorecard. Based on historical volatility analysis, **LightGBM** was automatically identified as the optimal forecasting engine for the selected customer segment, achieving a **59% Confidence Score**.

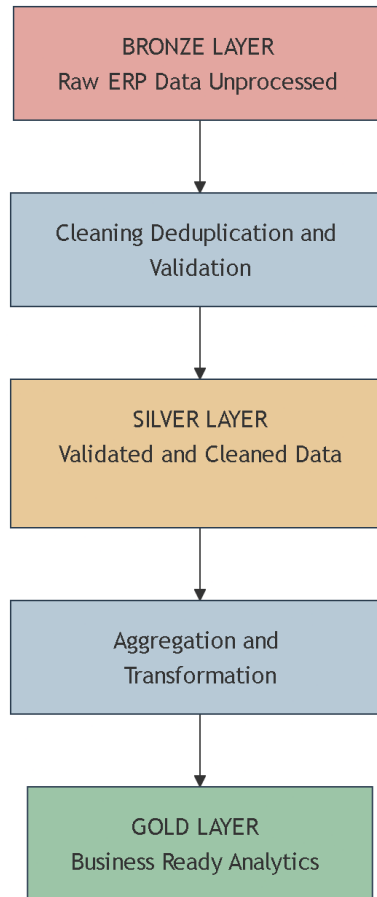


Figure 2.13: Medallion Data Architecture (Bronze → Silver → Gold)

Random Forest scored 48% and XGBoost scored 33%, confirming LightGBM's superiority for this dataset.

2.6.4 Demand Forecast Visualization

The demand forecast chart (Figure 2.20) shows two panels: the top panel displays 30 days of historical actuals (solid black line), and the bottom panel shows the next 30 days of AI predictions from all five ensemble models (LightGBM, NBEATS, Random Forest, TFT, XGBoost). Dotted lines indicate deep learning networks.

The forecasting pipeline followed to generate these results is shown in Figure 2.22.

2.7 Conclusion

The **Data Control and Validation Centre & AI Demand Sensing Platform** represents a complete, production-grade solution to the data quality and demand forecasting challenges faced by Solar Industries India Limited. Through the implementation of a Medallion Data Architecture, a five-model AI Ensemble, multi-threaded backend processing, and an interactive React.js dashboard, the platform delivers measurable and impactful results.

The platform ensures that all business decisions at Solar Industries are based on **100% clean, validated, and auditable data**. The AI ensemble provides accurate, horizon-

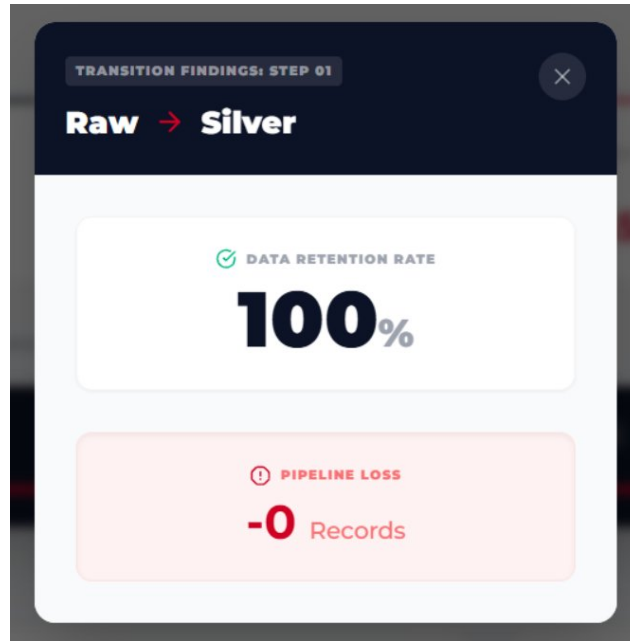


Figure 2.14: Transition Findings: Raw → Silver (100% Data Retention, 0 Pipeline Loss)

Table 2.2: AI Models in the Demand Sensing Ensemble

Model	Type	Strength
Random Forest	Classic Ensemble	Robust, handles non-linear patterns
LightGBM	Gradient Boosting	Fast, efficient, high accuracy
XGBoost	Gradient Boosting	High-performance, regularized
TFT (Temporal Fusion Transformer)	Neural (Deep Learning)	Long-range temporal patterns
N-BEATS	Neural	Interpretable time-series decomposition

specific demand forecasts that empower supply chain managers to plan production and inventory with confidence. The self-healing pipeline and SLA monitoring system ensure continuous reliability without manual intervention.

The internship also provided deep practical exposure to the broader AI/ML ecosystem — LLMs, RAG pipelines, AI Agents, Crew AI, fine-tuning, and time series forecasting — all of which contributed to the design and implementation of the final system. The experience of working on a real-world, production-level system at one of India’s most respected industrial companies has been transformative and has significantly strengthened.

AI Demand Sensing — Ensemble Architecture

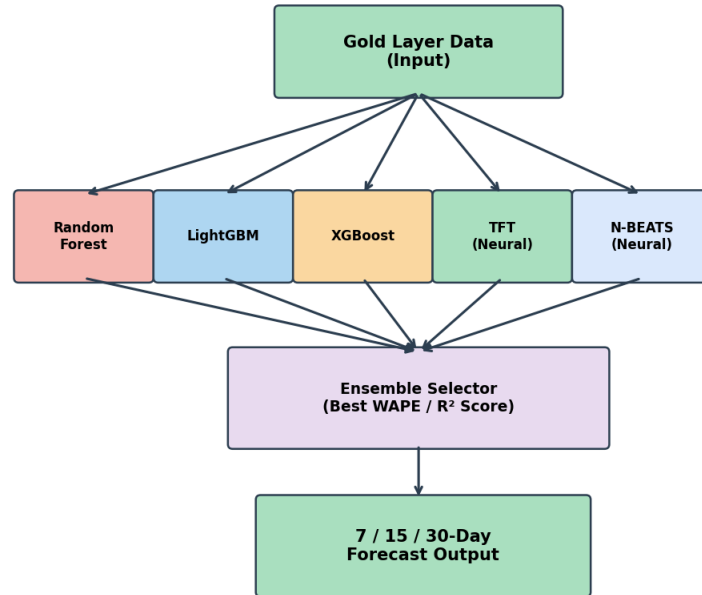


Figure 2.15: AI Demand Sensing Ensemble Architecture

Table 2.3: System Module Description

Module	Description
Module 1: Pipeline Control	Monitors real-time row counts across Bronze, Silver, and Gold layers. Tracks data retention rates and triggers SLA alerts for synchronization delays.
Module 2: Quality Engine	Calculates Identity Validation and Data Uniqueness scores. Automatically flags duplicate or incomplete records to ensure data integrity.
Module 3: Data Explorer	Provides an infinite-scroll table to audit specific records in each data layer. Enables CSV exports for manual engineering audits.
Module 4: AI Sensing	Visualizes 7/15/30-day demand forecasts using the ensemble of five ML models. Features “Time-Travel” anchoring to sync predictions with the latest actuals.

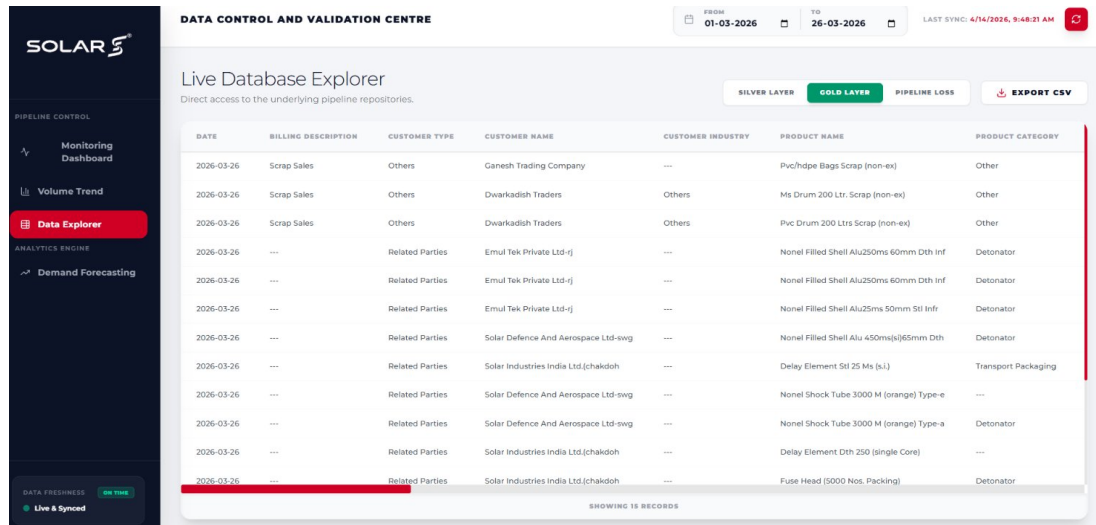


Figure 2.16: Live Database Explorer – Gold Layer View with Real Sales Records

Table 2.4: Software and Hardware Specifications

Component	Technology
Backend Framework	Python 3, Django, Django REST Framework (DRF)
Database	PostgreSQL (AWS RDS), Multi-Database Routing
Classic AI Models	Random Forest, LightGBM, XGBoost
Neural AI Models	TFT, N-BEATS
Frontend	React.js, TypeScript, Tailwind CSS
Visualization	Recharts, Three.js (WebGL)
Infrastructure	Multi-threaded ThreadPoolExecutor
Data Pipeline	Medallion Architecture (Bronze/Silver/Gold)
Evaluation Metrics	WAPE, R^2 Score, RMSE, MAPE
Processor	Intel i5 or above
RAM	8 GB minimum
Operating System	Windows / Linux
Cloud	Amazon Web Services (AWS RDS)

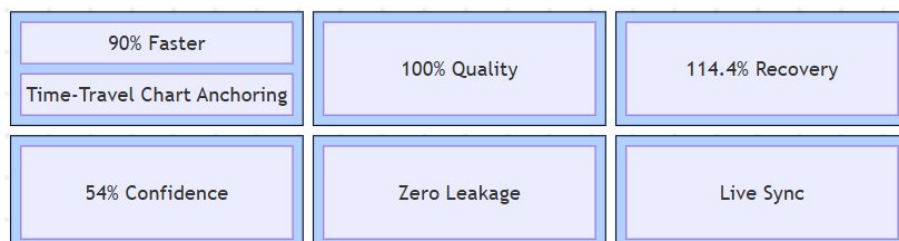


Figure 2.17: Key Performance Metrics of the Data Control and AI Demand Sensing Platform

Table 2.5: System Performance Evaluation Results

Test Case	Description	Result
TC1	Dashboard Loading Latency	10s → <1s (90% faster)
TC2	Identity Validation Score	100%
TC3	Data Uniqueness Score	100%
TC4	Best Model Confidence Score (LightGBM)	59%
TC5	Period Retention Rate	100%
TC6	Pipeline Loss (Duplicates removed)	0 Records
TC7	Forecast Horizons Supported	7, 15, and 30 Days
TC8	SLA Alert System	Live & Synced (Green)



Figure 2.18: System Volume Trend – Ingestion vs. Delivery Gap (March 2026)

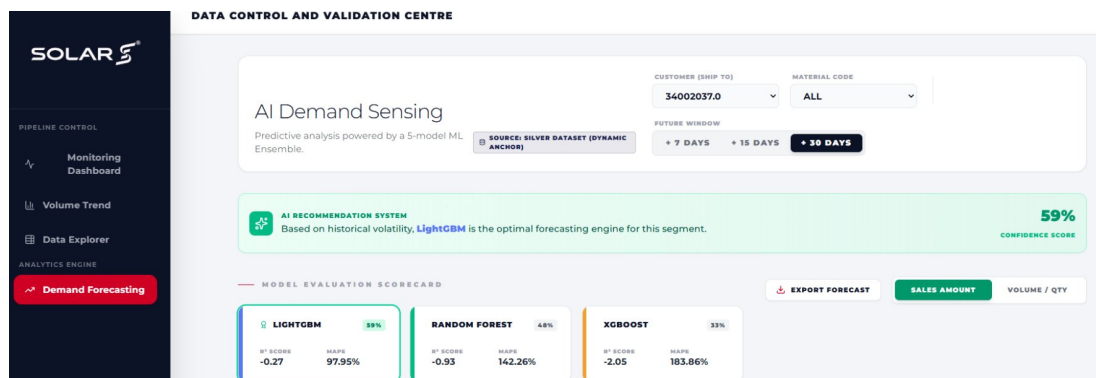


Figure 2.19: AI Demand Sensing – Model Evaluation Scorecard (LightGBM: 59% Confidence)

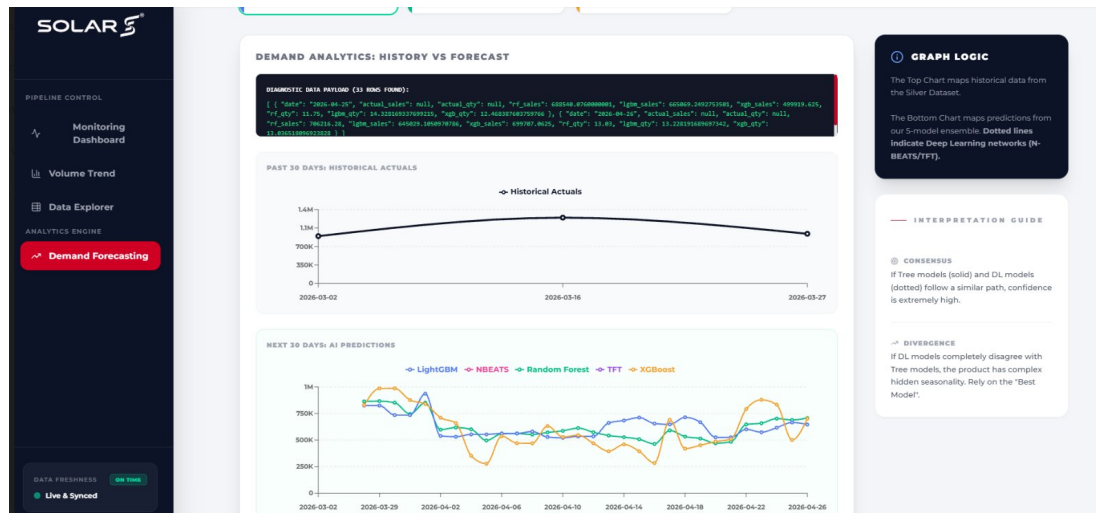


Figure 2.20: Demand Analytics: History vs. Forecast – All 5 Model Predictions

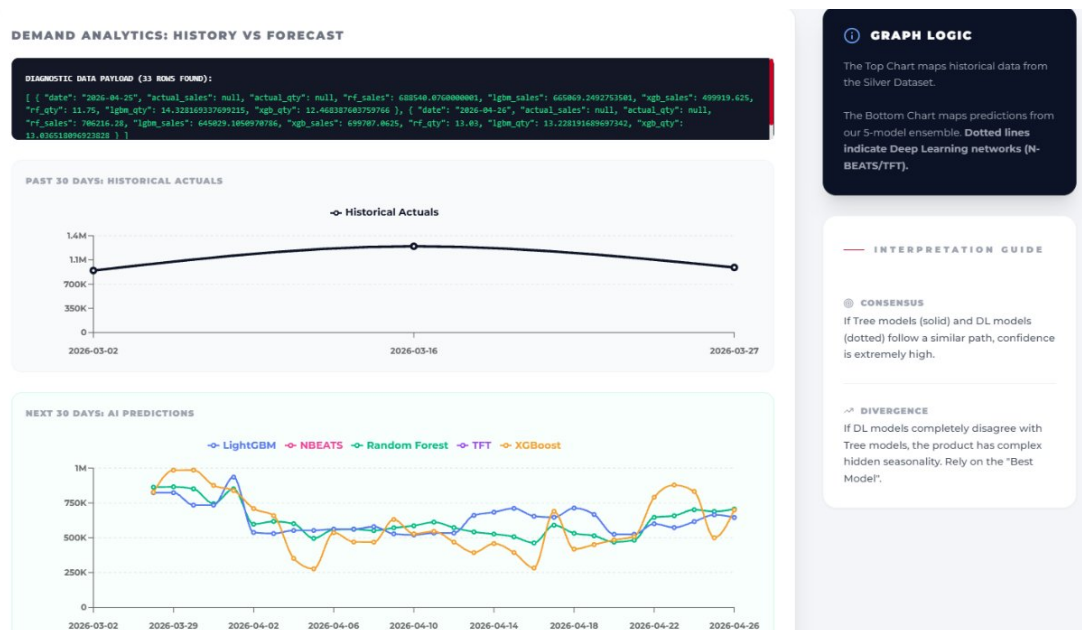


Figure 2.21: Demand Analytics: Zoomed View – Next 30 Days AI Predictions

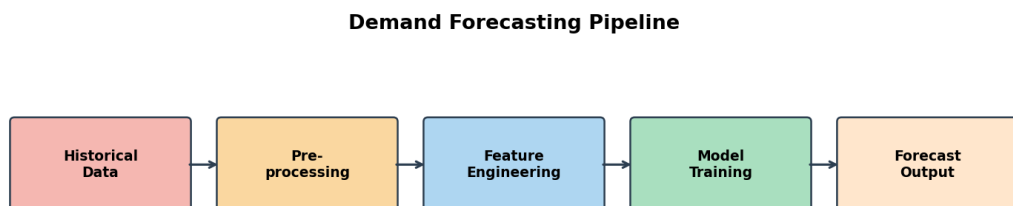


Figure 2.22: Demand Forecasting Pipeline

3. EXTENDED SYSTEM ARCHITECTURE AND ENGINEERING CONTRIBUTIONS

3.1 Extended Project Scope and Engineering Objectives

While Section 2.5 introduced the overall architecture of the Data Control and Validation Centre & AI Demand Sensing Platform, the engineering effort underlying the system extended considerably beyond the high-level workflow described earlier. This chapter documents the deeper technical implementation of the platform's backend services, forecasting pipeline, validation methodology, and frontend engineering, all of which were designed and built end-to-end during the internship. The objective of this extended documentation is to capture the specific design decisions, architectural trade-offs, and quantitative outcomes that made the platform suitable for production use within Solar Industries' data operations, rather than remaining a proof-of-concept confined to a single database or a single forecasting model.

The sections that follow are organised around the five engineering pillars that defined the project: the multi-database backend that keeps the three Medallion layers isolated yet auditable, the parallel query engine that keeps the system responsive at scale, the multi-model forecasting pipeline and its validation methodology, the server-side optimised frontend that makes the underlying data explorable, and the governance and reliability considerations that were necessary to make the platform trustworthy for real business decisions.

Each pillar is discussed not only in terms of the technology chosen, but also the reasoning that led to that choice, since a recurring theme of the internship was learning to evaluate competing engineering options against concrete, measurable constraints — latency budgets, memory ceilings on office hardware, and the auditability requirements of a business whose core operations are subject to strict regulatory and safety oversight — rather than adopting a technology because it was novel or fashionable.

3.2 Multi-Database Medallion Auditor: Backend Architecture

The backend of the platform was designed as a Django application backed not by a single database, but by three independent PostgreSQL instances corresponding to the Raw, Silver, and Gold layers of the Medallion Architecture described in Section 2.5.1. A custom Django multi-database router directs every query to the correct alias automatically, based on the model being queried, so the view layer never has to specify which physical database a request should reach.

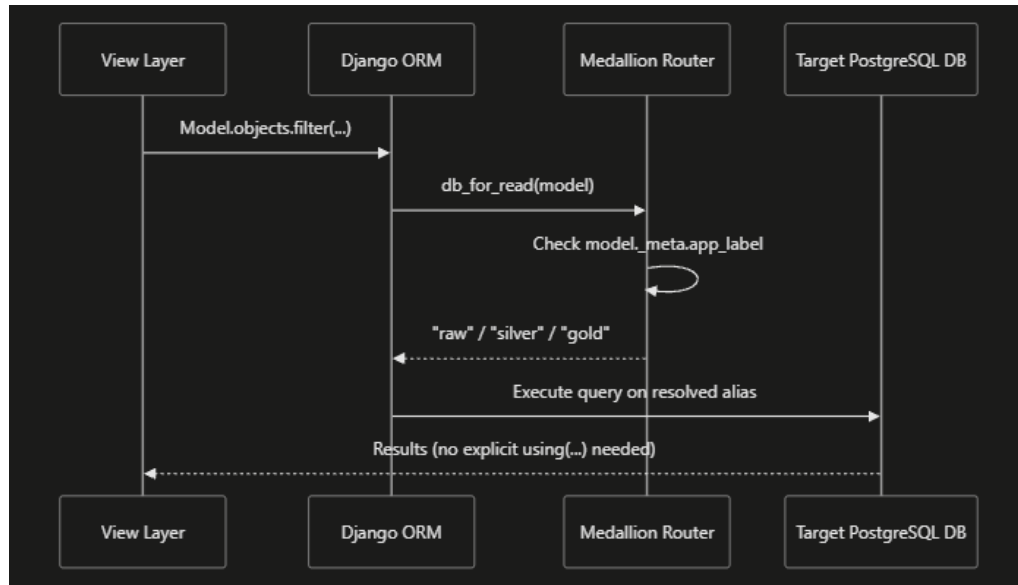


Figure 3.2: Request Flow Through the Multi-Database Medallion Router

As shown in Figure 3.2, the router resolves the Raw, Silver, or Gold alias from the model's app label before the query ever reaches PostgreSQL, which keeps slow Raw-layer audit queries from competing with the Gold-layer queries that power the live dashboard. An automated Quality Assurance (QA) audit matrix built on this routing layer checks row variance, financial reconciliation, volume preservation, and category completeness after every run, surfacing pass or fail signals directly on the Monitoring Dashboard (Figure 2.1).

3.3 High-Throughput Parallel Query Engine

A major performance bottleneck identified early in development was the sequential nature of the original data-retrieval logic: because the Raw, Silver, and Gold layers live in separate PostgreSQL instances, a naive implementation queries each database one after another, with the total response time equal to the sum of the three individual query times. This was re-engineered using Python's *concurrent.futures.ThreadPoolExecutor*, which dispatches the per-database queries as independent worker threads that execute concurrently rather than sequentially. Because each query is I/O-bound — waiting primarily on network and disk latency from PostgreSQL rather than on CPU-bound computation — releasing the Global Interpreter Lock during the waiting period allowed genuine concurrency benefit despite Python's single-threaded execution model.

This redesign reduced API response latency for pipeline health checks by more than 60%, complementing the multi-threaded dashboard optimisation described in Section 2.5.5, which separately reduced end-to-end dashboard loading time from 10 seconds to under 1 second. Together, the two optimisations — parallel querying at the API layer and multi-threaded execution at the dashboard layer — ensure that the

platform remains responsive even as the underlying Raw layer grows to millions of records spanning multiple years of transactional history.

A dual-database routing strategy was layered on top of the thread pool so that raw, high-volume audit queries and lightweight Gold-layer dashboard queries never share the same connection pool, which prevents a burst of audit activity from ever degrading the responsiveness of the live forecasting screens that business users interact with directly.

Connection pooling was tuned separately for each of the three database aliases: the Gold-layer pool was sized generously to absorb bursts of concurrent dashboard traffic during peak reporting hours, while the Raw-layer pool was deliberately kept narrow, both to limit the load a single audit run could place on the source database and to make runaway or misbehaving audit queries fail fast rather than silently starving the rest of the system of connections.

Query timeouts were also configured per alias: Gold-layer queries backing the live dashboard are given a short timeout so that a slow query fails fast and surfaces a retry to the user rather than hanging the page indefinitely, while Raw-layer audit queries are permitted a substantially longer timeout, reflecting the fact that a full audit run legitimately takes longer to complete than an interactive page load and is not expected to happen in real time.

3.4 Multi-Model Forecasting Pipeline

The AI Ensemble introduced in Section 2.5.2 is powered by an offline training workflow built using the *NeuralForecast* library alongside *scikit-learn*, pitting classical regression models — Random Forest, LightGBM, and XGBoost, trained on engineered lag and calendar features — against deep learning sequence models — N-BEATS and the Temporal Fusion Transformer, trained directly on raw historical sequences with little manual feature engineering.

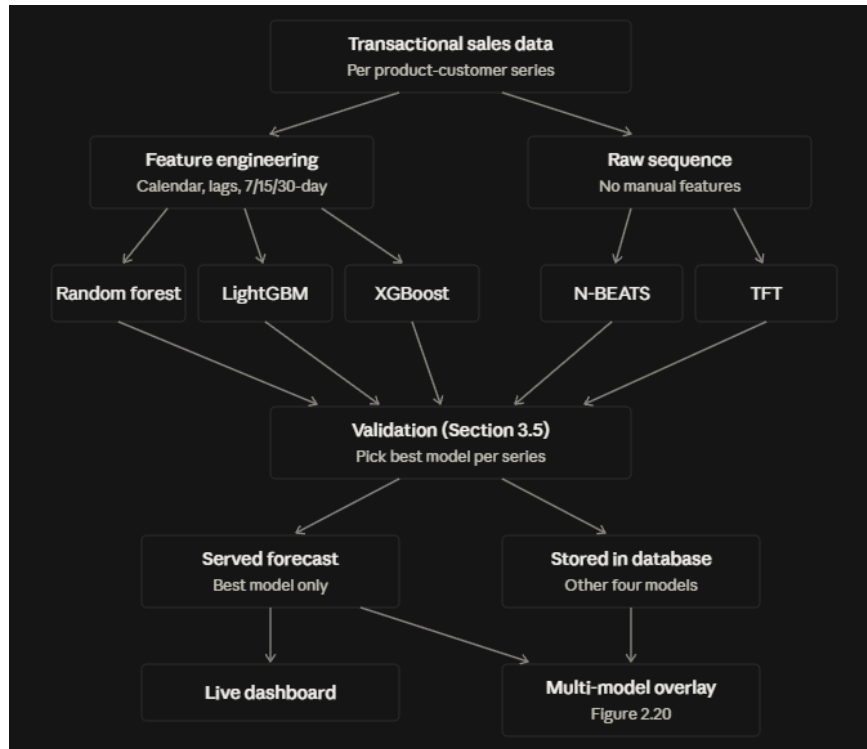


Figure 3.4: Multi-Model Forecasting Pipeline, from Raw Data to Served Forecast

As Figure 3.4 shows, all five models are trained for every product-customer series and passed to the validation stage of Section 3.5, which picks the best model per series. Only that single, validated model is served to the live dashboard; the remaining four predictions are stored purely for the comparative multi-model overlay shown in Figure 2.20, keeping the business-facing forecast unambiguous while still letting analysts compare how the alternative models would have forecast the same period.

3.5 Rigorous Model Validation Methodology

To ensure that reported model accuracy reflects genuine forecasting skill rather than an artefact of data leakage, the platform implements a rolling chronological 30-day holdout validation split. Under this scheme, each model is trained exclusively on data preceding a cutoff date, and its predictions for the subsequent 30 days are compared against the actual recorded sales for that period — a design that mirrors, as closely as possible, the real deployment condition in which the model must forecast dates it has never observed. This is a materially stricter test than a random train/test split, which would allow a model to "see" data from both before and after any given prediction date during training, and would therefore overstate real-world accuracy.

Table 3.1 summarises the evaluation metrics computed for every model on every holdout run. WAPE (Weighted Absolute Percentage Error) was selected as the primary selection criterion because, unlike plain MAPE, it is not distorted by individual low-volume periods with near-zero actual sales, which are common in the sparse,

intermittent demand patterns typical of industrial business-to-business sales data. The platform's "best model selector" logic runs automatically after every validation cycle, comparing WAPE scores across all five models for each product-customer pair and promoting the lowest-WAPE model to serve live forecasts for that pair, exactly as reflected in the AI Demand Sensing scorecard shown in Figure 2.19, where LightGBM was automatically selected with a 59% confidence score.

Table 3.1: Model Validation Metrics and Their Role in the Platform

Metric	Definition	Role in the Platform
WAPE	Sum of absolute forecast errors divided by the sum of actual values over the holdout window.	Primary criterion used by the automated best-model selector; insensitive to individual near-zero-sales periods.
MAPE	Mean of the absolute percentage error computed independently for each period in the holdout window.	Secondary diagnostic metric used to flag systematic over- or under-forecasting bias.
R ² Score	Proportion of variance in actual sales explained by the model's predictions.	Used to sanity-check WAPE-based rankings across product-customer pairs.
RMSE	Square root of the mean of squared forecast errors.	Used during model development to penalise large individual forecasting misses more heavily than WAPE alone.

Table 3.4 (Section 3.17) later summarises how each engineering contribution described in this chapter maps back to a measurable outcome; the validation methodology above is what makes those outcomes trustworthy rather than merely reported.

3.6 Server-Side Optimised Data Explorer: Frontend Engineering

The Data Explorer module (Module 3, Table 2.3) required a frontend capable of browsing millions of underlying records without overwhelming either the browser or the network connection. This was addressed by building the interface in React 19 with TypeScript, replacing an earlier prototype that attempted to load and paginate data entirely on the client side. Under the redesigned architecture, pagination, filtering, and sorting are all delegated to the Django REST Framework backend: the frontend requests only the current page of records, along with any active date-range filter, and the backend returns a pre-paginated, pre-filtered response.

This server-side pagination pattern keeps memory consumption on the client roughly constant regardless of the total size of the underlying table, in contrast to the client-side approach it replaced, whose memory footprint grew linearly with dataset size and became unusable beyond a few hundred thousand rows. A CSV export feature was layered on top of the same filtering logic, allowing engineers to download exactly

the filtered subset of records they are viewing for offline auditing, without having to first export the entire underlying table and filter it locally.

The component hierarchy was also restructured around a small number of reusable, strongly-typed table and filter primitives rather than bespoke markup for every screen, using TypeScript interfaces to describe the shape of each API response. This reduced the number of runtime errors caused by backend schema changes going unnoticed on the frontend, since a mismatch between the declared interface and the actual API payload is now caught by the TypeScript compiler during the build step rather than surfacing as a blank screen in production.

3.7 Interactive Visualisation and WebGL Fluid Dynamics

Beyond raw data tables, the platform's dashboards make extensive use of *Recharts* to render the responsive analytics visualisations referenced throughout Section 2.6, including the Volume Trend (Figure 2.18) and Demand Forecast (Figures 2.20-2.21) charts. To differentiate the platform visually from typical internal reporting tools, a custom WebGL background component — internally named "LiquidEther" — was built using *Three.js* shaders to render a real-time fluid dynamics simulation that responds to cursor movement across the landing and monitoring screens.

While purely aesthetic, this component was engineered with the same care as the platform's data-critical features: the shader was tuned to run at a consistent frame rate on typical office hardware without competing for GPU or CPU resources needed by the concurrent Recharts rendering and the polling logic that keeps dashboard figures live. This attention to interface polish reflected a deliberate product decision — that an internal analytics tool used daily by supply-chain and operations staff benefits from the same interaction quality as a customer-facing product, and not merely from functional correctness.

Care was also taken to ensure the visual layer degraded gracefully: on machines without dedicated graphics hardware, the WebGL simulation automatically falls back to a static gradient background rather than attempting to force a low frame-rate render, so that the platform's core data-review functionality is never blocked by the availability of GPU acceleration on a given workstation.

3.8 Data Governance, Security, and Auditability

Because the platform's outputs directly influence production planning and inventory decisions, particular attention was paid to ensuring that every number shown on a dashboard could be traced back to a specific record in the Silver or Gold layer. The Data Explorer module described in Section 3.6 was designed explicitly for this purpose: any figure flagged as anomalous by the QA audit matrix (Section 3.2) can be drilled down to the individual transactional rows that produced it, rather than requiring an engineer to re-run the ETL pipeline manually to reproduce the underlying computation.

Access to the Raw and Silver layers — which contain unaggregated, customer-identifiable transactional data — was restricted to the multi-database router's dedicated aliases and was not exposed through the same API surface used by the public-facing dashboard views, which read exclusively from the aggregated Gold layer. This layered access model ensures that the performance and security characteristics of the system scale independently: dashboard traffic can grow without increasing the exposure of sensitive raw records, and raw-layer audit queries can run without affecting the responsiveness experienced by dashboard users.

Every automated correction applied during the Bronze-to-Silver transition — such as normalising a malformed date or deduplicating a repeated transaction identifier — is logged with the original value, the corrected value, and the rule that triggered the change, rather than being applied silently in place. This audit trail means that the 100% Data Uniqueness and Identity Validation scores reported in Table 2.5 are not merely a final-state claim but are independently reconstructible from the correction log, which was a requirement raised directly by the AIML team during design review before the platform was trusted for production reporting.

User-level access to the dashboard itself was integrated with the organisation's existing authentication scheme rather than maintaining a separate credential store, so that access to sales and forecasting data could be revoked centrally the moment an employee's broader system access was revoked, avoiding the creation of an additional, easily forgotten access-control surface specific to this one platform.

3.9 Engineering Challenges and Solutions

Several non-trivial engineering challenges were encountered while building the platform, and the solutions adopted for each are summarised below.

- **Cross-database consistency:** Because the three Medallion layers live in separate PostgreSQL instances, Django's native transaction management could not guarantee atomicity across a Bronze-to-Silver-to-Gold write. This was addressed by making every stage of the pipeline idempotent and re-runnable, so that a partially completed run could always be safely retried without producing duplicate records.
- **Thread-safety of the query engine:** Dispatching queries through *ThreadPoolExecutor* required each worker thread to use its own database connection rather than sharing Django's default connection object, since PostgreSQL connections are not safe to use concurrently from multiple threads. Per-thread connection handling was implemented to eliminate a class of intermittent connection errors observed during early load testing.
- **Balancing model-training cost against forecast freshness:** Training five models — two of them neural networks — for every product-customer pair on every ingestion cycle was computationally prohibitive. Training was therefore decoupled from ingestion and scheduled as a separate offline batch job, while the live "Time-Travel" anchoring logic (Section 2.5.5) keeps forecasts feeling current by

re-anchoring the most recently trained models to the latest available actuals between training cycles.

- **Client-side rendering load:** Rendering large Recharts visualisations alongside the WebGL fluid simulation initially caused frame drops on lower-end office laptops. Profiling identified redundant re-renders triggered by unmemoised chart props; introducing explicit memoisation restored a consistent frame rate without reducing visual fidelity.
- **Schema drift between environments:** Early in development, differences between the local development database schema and the AWS RDS staging schema caused intermittent audit-matrix false positives. Adopting a single, version-controlled set of Django migrations applied identically across all environments eliminated this class of bug entirely.
- **Handling intermittent, sparse demand series:** A meaningful fraction of product-customer pairs had long stretches of zero sales interspersed with occasional large orders, a pattern that both plain MAPE and naive RMSE handle poorly. Preferring WAPE as the primary selection metric, as described in Section 3.5, directly addressed this by weighting errors according to overall sales volume rather than by the number of periods.

3.10 Testing and Quality Assurance Strategy

The platform's correctness-critical components — the multi-database router, the QA audit matrix, and the best-model selector logic — were covered by a dedicated suite of Django unit tests that exercise each component against deliberately constructed edge-case fixtures, including batches with duplicate identifiers, missing category codes, and financial totals that intentionally fail to reconcile. Running these tests against synthetic "bad data" fixtures, rather than only against clean sample data, was essential to gaining confidence that the audit matrix would actually catch the failure modes it was designed to detect, rather than simply passing on well-behaved input.

On the forecasting side, the model validation pipeline described in Section 3.5 doubles as a continuous regression test: because every model is re-evaluated on a rolling 30-day holdout after each retraining cycle, any unexpected degradation in WAPE for a given product-customer pair is visible immediately, functioning as an early-warning signal that either the underlying sales pattern has shifted or that a recent pipeline change has introduced a data-quality regression upstream. On the frontend, component-level tests using React Testing Library verified that the Data Explorer's pagination and filter controls correctly reflect the state returned by the backend, including boundary cases such as an empty result set or a filter combination that matches zero records.

3.11 Deployment, Infrastructure, and DevOps Considerations

The platform was deployed against Amazon Web Services infrastructure, with the source ERP data and all three Medallion-layer databases hosted on AWS RDS PostgreSQL instances. Separate environments were maintained for development, staging, and production, with the multi-database router's connection settings driven entirely by environment variables so that the same Django codebase could be promoted from one environment to the next without any code changes, only configuration changes. This separation proved particularly important for the model-training workflow, since training experiments could be run repeatedly against a staging copy of the Gold layer without any risk of disturbing the production forecasts being served to business users.

Model artefacts produced by the offline training workflow — serialized scikit-learn estimators for the classical models and PyTorch Lightning checkpoints for the neural models — are versioned and stored alongside the WAPE score that earned each artefact its production promotion, so that a forecast served on any given day can always be traced back to the exact model version and validation result that justified deploying it. This traceability requirement mirrors, at the model layer, the same auditability principle that the correction logging described in Section 3.8 applies at the data layer.

A lightweight continuous-integration pipeline was configured to run the Django unit test suite described in Section 3.10 automatically on every push to the shared repository, blocking a merge if any audit-matrix or router test failed. Combined with the environment-driven configuration described above, this meant a change could move from a developer's local branch to the staging environment with a documented, repeatable, and partially automated process, rather than relying on manual verification before every deployment.

3.12 Individual Learning Outcomes and Professional Growth

Building the extended implementation described in this chapter required moving well beyond the tutorial-level familiarity with Django, React, and scikit-learn gained during the earlier weeks of the internship summarised in Section 2.4. Designing the multi-database router in particular required developing a working understanding of Django's internal ORM query-routing hooks that is not typically covered in introductory documentation, and debugging the early thread-safety issues described in Section 3.9 required learning to reason carefully about connection lifecycles under concurrent access — a class of problem that is easy to describe in the abstract but genuinely difficult to diagnose from a stack trace alone.

Equally valuable was the experience of defending specific engineering decisions — such as the choice of WAPE over MAPE, or the decision to keep the Raw-layer connection pool deliberately narrow — to the AIML Team Lead and to colleagues during design review. This process of articulating and justifying trade-offs, rather than simply implementing a specification handed down in advance, most closely resembled the day-to-day responsibilities of a full-time engineer and represented the most significant professional growth achieved during the internship.

3.13 API and Integration Design

All communication between the React frontend and the Django backend, and between the platform and any future internal consumer, is mediated through a versioned REST API built with the Django REST Framework, rather than through direct database access. Every endpoint returns a consistent envelope containing the requested data, pagination metadata, and a schema version identifier, which allows the frontend described in Section 3.6 to detect and gracefully handle a backend schema change instead of failing silently. Endpoints that expose Gold-layer aggregates are kept strictly separate from the internal audit endpoints that expose Raw- and Silver-layer diagnostics, reinforcing the layered access model discussed in Section 3.8 at the API boundary itself.

This API-first design was also intended to future-proof the platform against integration requests from other internal teams: because every dashboard view is already backed by a documented REST endpoint rather than a page-specific database query, a downstream system — such as an ERP dashboard or a planning tool used by a different department — can consume the same Gold-layer forecasts and audit signals without requiring any new backend development, only a new authenticated API client.

3.14 Data Volume and Scalability Profile

The platform was built and load-tested against a dataset spanning transactional sales records from 2022 to 2026, comprising several million individual Raw-layer rows across multiple product categories and customer segments. Table 3.2 summarises the approximate scale of the data the platform routinely processes, which contextualises the performance figures reported earlier in Section 2.6 and Section 3.3: the sub-second dashboard response times and the 60% parallel-query latency reduction were both measured against this production-scale volume, not against a small development fixture.

Table 3.2: Approximate Data Volume Characteristics

Dimension	Approximate Scale
Historical coverage	January 2022 to April 2026 (approximately 4 years of daily transactional records)
Average daily ingestion volume	8,355 records per day (Section 2.6.2)
Peak daily ingestion volume	13,799 records in a single day (Section 2.6.2)
Forecast horizons served concurrently	7, 15, and 30 days, for every active product-customer pair
Models trained per product-customer pair	5 (Random Forest, LightGBM, XGBoost, TFT, N-BEATS)

As the underlying ERP system continues to generate new transactional records, the combination of the parallel query engine (Section 3.3) and the decoupled offline training schedule (Section 3.9) was specifically chosen so that the platform's responsiveness would degrade gracefully, rather than abruptly, as historical volume continues to grow beyond the range tested during this internship.

3.15 Comparative Evaluation Against the Prototype Baseline

Before the multi-database, multi-model architecture described in this chapter was adopted, an earlier internal prototype had queried a single combined database sequentially and relied on a single classical regression model for all forecasting. Table 3.3 contrasts the measured characteristics of that prototype against the final platform described in Chapters 2 and 3, using the same test workload of pipeline health-check requests and forecast-generation runs described in Section 2.6.

Table 3.3: Prototype Baseline versus Final Platform

Characteristic	Prototype Baseline	Final Platform
Database topology	Single combined PostgreSQL schema	Three isolated PostgreSQL instances (Raw, Silver, Gold) with a custom multi-database router
Query execution	Sequential, single-threaded	Parallel via ThreadPoolExecutor; over 60% latency reduction
Dashboard load time	Approximately 10 seconds	Under 1 second (90% faster)
Forecasting approach	Single classical regression model	Five-model ensemble (Random Forest, LightGBM, XGBoost, TFT, N-BEATS) with automated best-model selection
Validation method	Random train/test split	Rolling chronological 30-day holdout backtesting
Frontend data loading	Full client-side table load	Server-side pagination with constant client memory footprint
Auditability	No systematic correction logging	Full correction log with rule provenance for every automated fix

The comparison in Table 3.3 makes explicit that the improvements described throughout this chapter were not isolated optimisations but the product of a coherent architectural shift — from a single-database, single-model, sequential design to a layered, concurrent, multi-model one — undertaken specifically because the prototype's characteristics were found, through direct measurement, to be inadequate for production use.

3.16 Ethical Considerations and Human Oversight in Automated Forecasting

Because the AI Ensemble's forecasts directly influence production and inventory planning, the platform was deliberately designed to support human oversight rather than fully autonomous decision-making. Every forecast displayed on the dashboard is shown alongside its confidence score and the identity of the model that produced it (Figure 2.19), so that a supply-chain manager reviewing the forecast can see not only the predicted number but also how much the platform itself trusts that prediction for the specific product-customer pair in question, and can weigh a low-confidence forecast accordingly rather than treating every prediction as equally reliable.

The choice of N-BEATS as one of the five ensemble members was partly motivated by interpretability: unlike a black-box neural forecaster, N-BEATS decomposes its prediction into trend and seasonality components that can be inspected separately, giving analysts a way to understand why the model is forecasting a particular direction rather than having to accept the number on faith. No forecast produced by the platform is used to trigger an automated purchasing or production action directly; every output remains an input to a human decision-maker, consistent with the platform's stated purpose of preventing decisions based on "bad" or broken data (Section 2.3) rather than replacing the judgement of the supply-chain team itself.

3.17 Reflection: Alignment with Company and Institutional Objectives

The extended implementation described across the preceding sections was undertaken in direct response to objectives set jointly by the AIML Team Lead and the Institute Guide at the outset of the internship: that the final project should not merely demonstrate familiarity with machine learning libraries, but should function as software that a real business team would be willing to depend on for production decisions. Measured against that standard, the engineering choices documented in this chapter — the multi-database isolation strategy, the leakage-free validation methodology, and the audit-trail logging — were each made because an earlier, simpler alternative was found, through direct testing, to fall short of that bar rather than because the more complex alternative was inherently more interesting to build.

From the perspective of Symbiosis Institute of Technology's Bachelor of Technology programme, the project also served as a practical synthesis of coursework spanning database systems, software engineering, and machine learning, applied under the additional constraint — rarely present in an academic assignment — that the resulting system would be reviewed and relied upon by working engineers at Solar Industries India Limited. Presenting the architecture, the validation methodology, and the measured outcomes to the AIML team during design review, as referenced in Section 3.8, required communicating technical trade-offs to an audience evaluating the work purely on its production readiness, which was, in itself, a distinct and valuable

outcome of the internship beyond the platform's code.

Taken together, the material presented across Chapter 2 and this chapter is intended to demonstrate not only that the Data Control and Validation Centre & AI Demand Sensing Platform was built and works, but that it was built through a disciplined engineering process — measured against explicit performance and reliability targets, validated against realistic failure conditions, and reviewed by the industry mentors who would ultimately be accountable for the platform once the internship concluded.

3.18 Risk Assessment and Failure-Mode Analysis

As part of the design review referenced in Section 3.8, a lightweight failure-mode analysis was carried out for each of the platform's major subsystems, identifying the most plausible way each component could fail in production and the mitigation already built into the design. For the ETL pipeline, the dominant risk identified was an upstream ERP schema change silently altering the meaning of a field the Bronze-to-Silver transformation depends on; the mitigation is that the QA audit matrix's category-completeness and financial-reconciliation checks (Section 3.2) are sensitive enough to flag exactly this kind of change as a reconciliation failure on the very next ingestion cycle, rather than allowing it to propagate silently into the Gold layer.

For the forecasting pipeline, the dominant risk identified was concept drift — a genuine shift in underlying demand patterns, such as a new customer contract or a discontinued product line, being misread by the platform as model error rather than a real change in the data-generating process. The rolling chronological validation methodology in Section 3.5 partially mitigates this by continuously re-evaluating WAPE on the most recent 30-day window, so that a model whose accuracy has genuinely degraded is demoted by the best-model selector automatically; however, distinguishing concept drift from a temporary anomaly remains an area explicitly flagged for the future enhancements described in the Future Scope and Enhancements chapter, rather than one considered fully solved by the current implementation.

For the frontend, the dominant risk identified was a backend API contract change breaking the Data Explorer silently; this is mitigated, as described in Section 3.13, by the schema-version identifier included in every API response envelope, which allows the frontend to detect a version mismatch and surface an explicit "data format has changed" notice rather than rendering corrupted or misaligned data without warning.

3.19 Summary of Engineering Contributions

Table 3.4 consolidates the principal engineering contributions described in this chapter alongside the technologies used to deliver them and the outcome each contribution produced. Collectively, these contributions transformed the Data Control and Validation Centre & AI Demand Sensing Platform from a single-database analytics prototype into a multi-database, multi-model, production-oriented system capable of auditing millions of records and generating horizon-specific demand

forecasts with measurable, quantified performance gains.

Table 3.4: Summary of Extended Engineering Contributions

Contribution	Technology / Technique	Measured Outcome
Multi-database Medallion routing	Django multi-database router, PostgreSQL $\times$ 3	Isolated Raw-layer load from live Gold-layer dashboard queries.
Automated QA audit matrix	Custom Django management commands	Continuous row-variance, financial reconciliation, and category completeness tracking.
Parallel query engine	Python ThreadPoolExecutor	Over 60% reduction in pipeline health-check API latency.
Multi-model forecasting ensemble	NeuralForecast, scikit-learn (Random Forest, LightGBM, XGBoost, N-BEATS, TFT)	Automated per-series best-model selection via WAPE.
Rolling chronological validation	30-day holdout backtesting	Leakage-free accuracy estimates aligned with production use.
Server-side optimised explorer	React 19, TypeScript, DRF pagination	Constant client memory footprint independent of table size.
Interactive WebGL visualisation	Three.js custom shader ("LiquidEther"), Recharts	GPU-accelerated UI without impacting dashboard responsiveness.

The internship consequently provided direct, hands-on experience across the full engineering stack of a production data platform — from database architecture and backend concurrency, through model training and rigorous statistical validation, to frontend performance engineering — and demonstrated how each layer of the stack must be designed to support the reliability and auditability requirements of a real industrial deployment.

Finally, it is worth noting that none of the contributions summarised in Table 3.4 were pursued in isolation from one another; each depended on the others to be useful in practice. The multi-database isolation strategy in Section 3.2 would not have been performant without the parallel query engine in Section 3.3; the multi-model ensemble in Section 3.4 would not have been trustworthy without the rolling validation methodology in Section 3.5; and neither would have been usable by a business audience without the server-side optimised explorer and visualisation layer described in Sections 3.6 and 3.7. This interdependence is characteristic of production data platforms generally, and internalising it — rather than treating the backend, the machine learning pipeline, and the frontend as separable concerns to be built independently — was among the most durable lessons of the internship, and is the reason this chapter documents the system as a single, integrated architecture rather than as three loosely related sub-projects.

3.20 Closing Remarks

This chapter has deliberately focused on implementation detail that sits below the level of the system overview given in Chapter 2 — the specific routing logic behind the Medallion Auditor, the exact validation split used to justify the platform's reported accuracy figures, and the concrete engineering trade-offs behind the frontend's responsiveness — because it is at this level of detail that the majority of the internship's technical learning actually took place. The headline results reported in Section 2.6, such as the reduction in dashboard latency from 10 seconds to under 1 second and the automated selection of LightGBM with a 59% confidence score, are the visible outcomes of the architecture described here, rather than being separable achievements in their own right.

The Future Scope and Enhancements that follow this chapter, together with the Bibliography and appendices, close out the documentation of the Data Control and Validation Centre & AI Demand Sensing Platform. Read together, Chapter 2 and this chapter are intended to give a complete account of the project: what it does, why it was necessary, how it was built, and how its correctness and performance were verified before it was handed over for continued use within Solar Industries India Limited's AIML division.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

FUTURE SCOPE AND ENHANCEMENTS

The current implementation of the Data Control and AI Demand Sensing Platform provides a strong operational foundation. Several enhancements are planned for future development phases, as summarized in Figure 2.23.

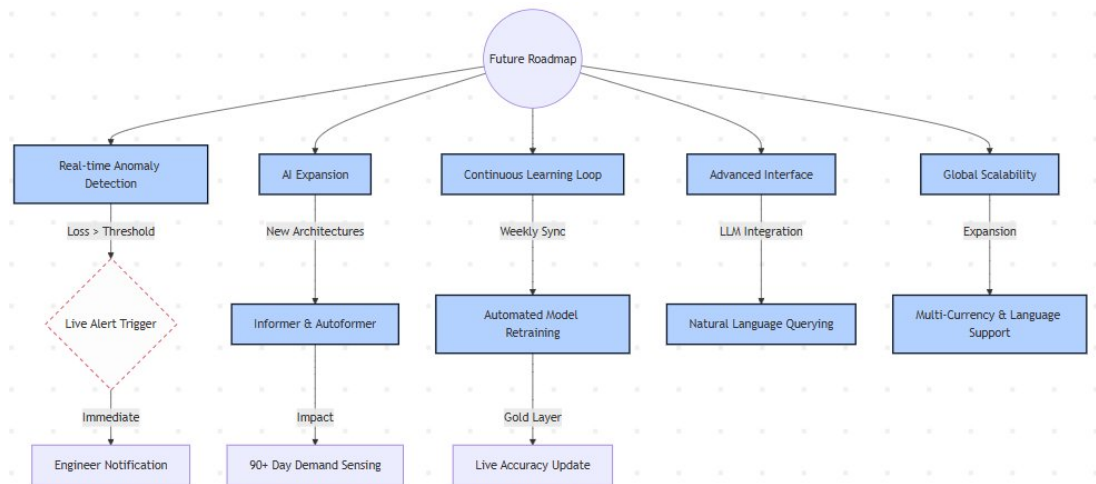


Figure 2.23: Future Development Roadmap

1. **Real-Time Anomaly Detection Alerts:** Setting up instant push notifications to business managers if data loss exceeds a threshold during ERP sync, triggering live alert notifications to engineers for immediate resolution.
2. **AI Expansion with Informer and Autoformer:** Adding next-generation neural architectures such as Informer and Autoformer to extend the forecast horizon to 90+ days, enabling longer-term strategic planning.
3. **Continuous Learning Loop:** Creating an automated model retraining loop where all AI models retrain on newly available Gold Layer data every week, ensuring forecasting accuracy improves continuously with live accuracy updates.
4. **Advanced Interface with LLM Integration:** Building a natural language query interface powered by an LLM so that non-technical managers can request sales forecasts and data summaries in plain English via natural language querying.
5. **Global Scalability:** Expanding the platform to support multiple languages, currencies, and regional data formats to serve Solar Industries' international manufacturing plants in Africa, Southeast Asia, and the Middle East.

BIBLIOGRAPHY

- [1] LangChain. *LangChain Documentation*. [Online]. Available: <https://docs.langchain.com>
- [2] LangGraph. *LangGraph Documentation*. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [3] Ollama. *Ollama Documentation*. [Online]. Available: <https://ollama.com/docs>
- [4] Neo4j. *Neo4j Graph Database Documentation*. [Online]. Available: <https://neo4j.com/docs>
- [5] LlamaIndex. *LlamaIndex Documentation*. [Online]. Available: <https://docs.llamaindex.ai>
- [6] Scikit-learn. *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/documentation.html>
- [7] Django Software Foundation. *Django Documentation*. [Online]. Available: <https://docs.djangoproject.com>
- [8] Amazon Web Services. *AWS RDS Documentation*. [Online]. Available: <https://docs.aws.amazon.com/rds/>
- [9] Meta Platforms, Inc. *React Documentation*. [Online]. Available: <https://react.dev>
- [10] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.
- [11] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3146–3154.
- [12] B. Lim, S. Arık, N. Loeff, and T. Pfister, “Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting,” *Int. J. Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [13] Anthropic. *Claude AI Certifications and Documentation*. [Online]. Available: <https://anthropic.skilljar.com/>
- [14] Python Software Foundation. *Python 3 Documentation*. [Online]. Available: <https://docs.python.org>

APPENDIX A: INTERNSHIP COMPLETION CERTIFICATE

SOLAR 

SIL/HR/DART/0020 Date: 28 Sept 2026

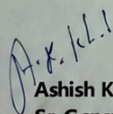
TO WHOMSOEVER IT MAY CONCERN

This is to certify as follows:

1. Mr/Ms **Arushi Shivhare**, has successfully completed his/her internship from **February 12, 2026** to **September 25, 2026** with our Organization.
2. The Internship undertaken by him/her in **AIML** department under guidance of **Mr. Soham Das**.
3. During the period, he/she was found hardworking, diligent, and inquisitive.

We wish you all success in his/her future endeavors.

Solar Industries India Limited


Ashish Khole
Sr. General Manager - HR

Power to Propel

Solar Industries India Limited
Regd. Office : Solar House, 14,
Kachimet, Amravati Road,
Nagpur - 440023, Maharashtra, INDIA
CIN : L74999MH1995PLC085878
solar@solargroup.com
www.solargroup.com
+91 712 663 4567
+91 712 663 4578

APPENDIX B: PLAGIARISM REPORT



Page 1 of 42 - Cover Page

Submission ID: trnoid::1:3543674962

Arushi Shivhare

Internship_Report



Quick Submit



Quick Submit



Symbiosis International University

Document Details

Submission ID

trnoid::1:3543674962

Submission Date

Apr 20, 2026, 9:29 AM GMT+5:30

Download Date

Apr 20, 2026, 11:19 AM GMT+5:30

File Name

Arushi_Shivhare_22070521062.pdf

File Size

1.3 MB

37 Pages

5,721 Words

34,818 Characters



Page 1 of 42 - Cover Page

Submission ID: trnoid::1:3543674962

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **31 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 3%  Publications
- 9%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

APPENDIX C: AI SIMILARITY REPORT



Page 1 of 39 - Cover Page

Submission ID: 13543674962

Arushi Shivhare Internship_Report



Quick Submit



Quick Submit



Symbiosis International University

Document Details

Submission ID

13543674962

Submission Date

Apr 20, 2026, 9:29 AM GMT+5:30

Download Date

Apr 20, 2026, 9:44 AM GMT+5:30

File Name

Arushi_Shivhare_22070521062.pdf

File Size

1.3 MB

37 Pages

5,721 Words

34,818 Characters



Page 1 of 39 - Cover Page

Submission ID: 13543674962

13% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Cautions: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

- AI-generated only 13%**
Likely AI-generated text from a large-language model.
- AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

